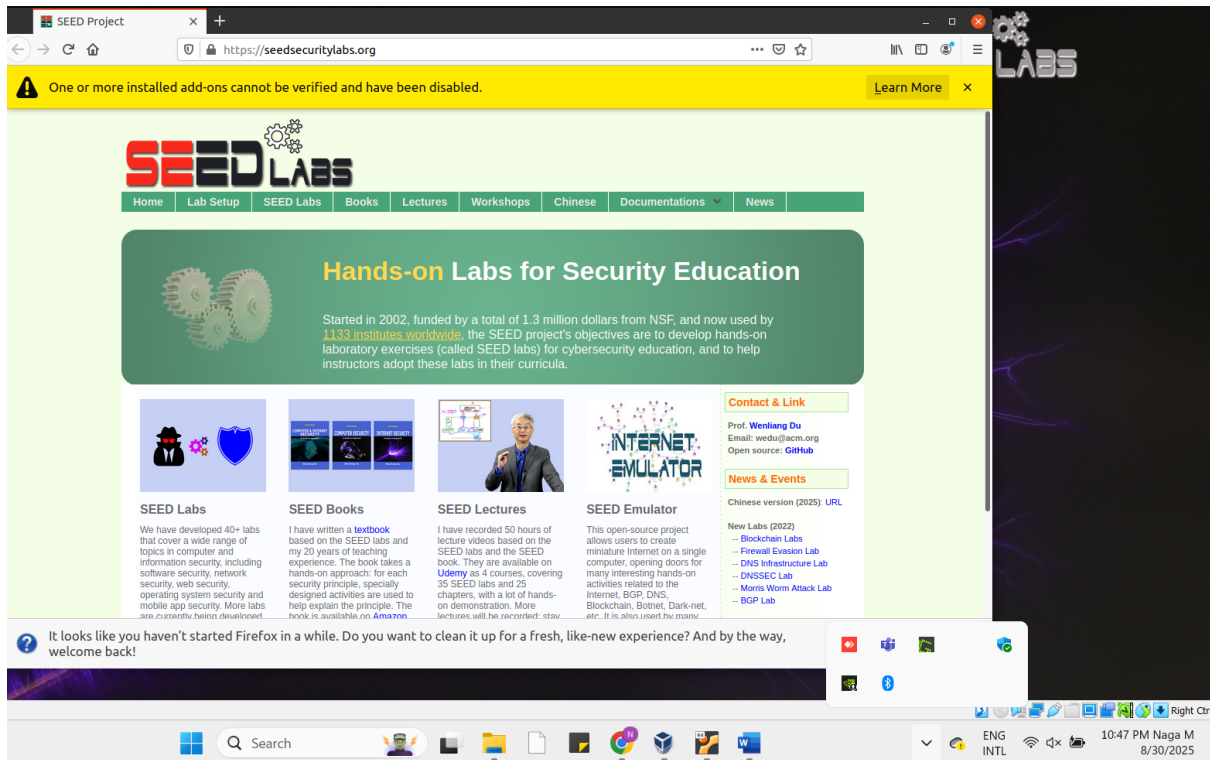


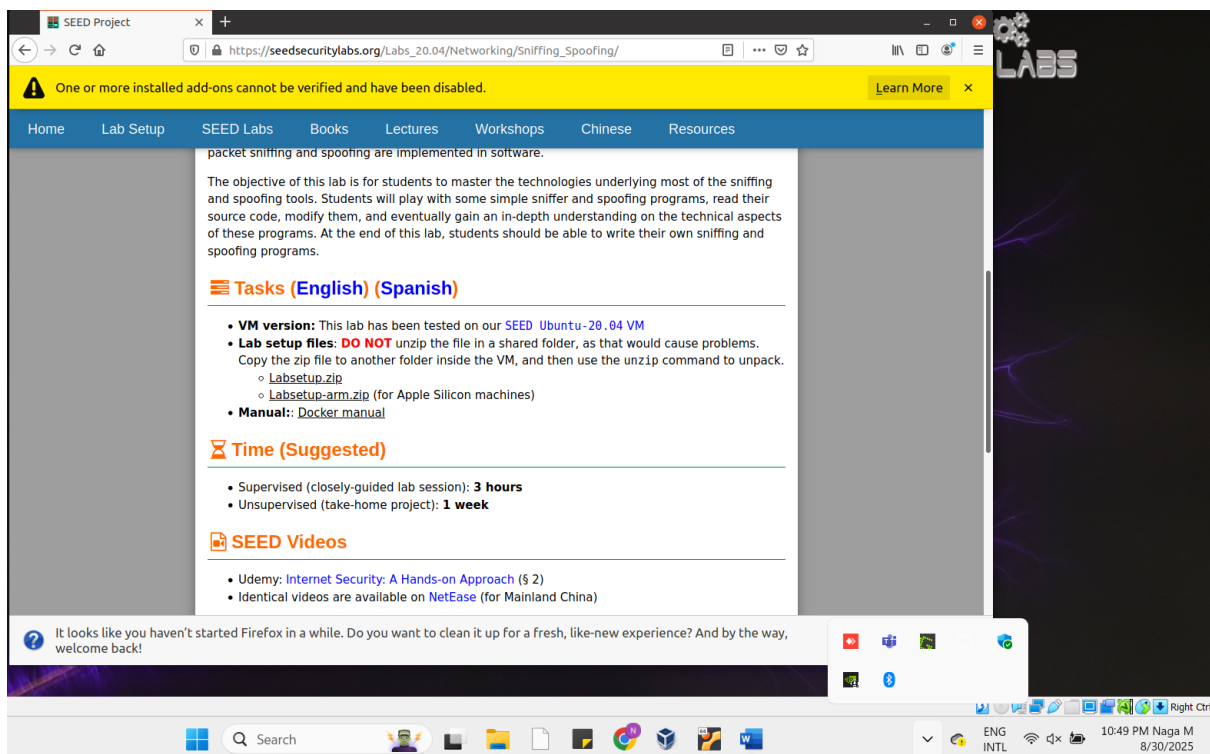
Packet Sniffing and Spoofing

LAB Setup Environment

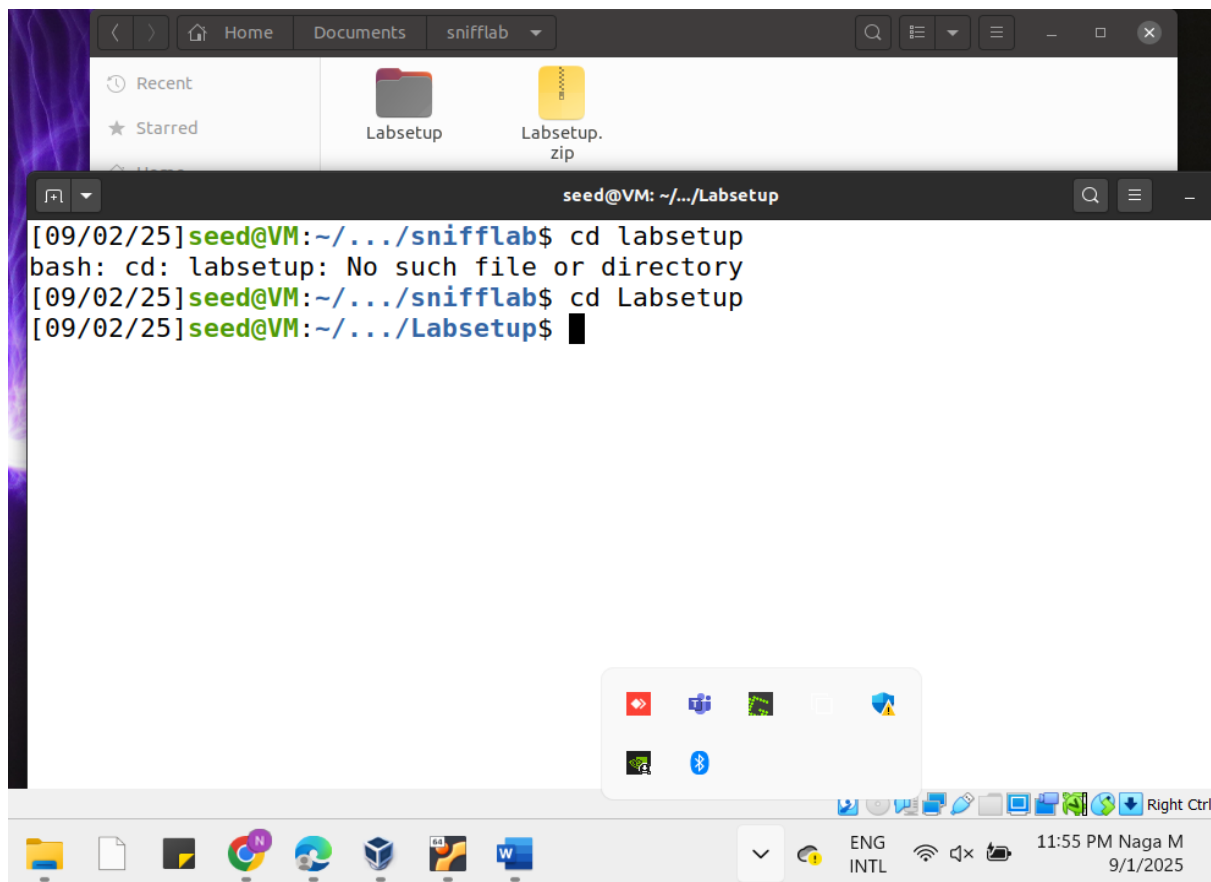
1. Run the SEED VM>> open Firefox in that>> go to the seed labs,



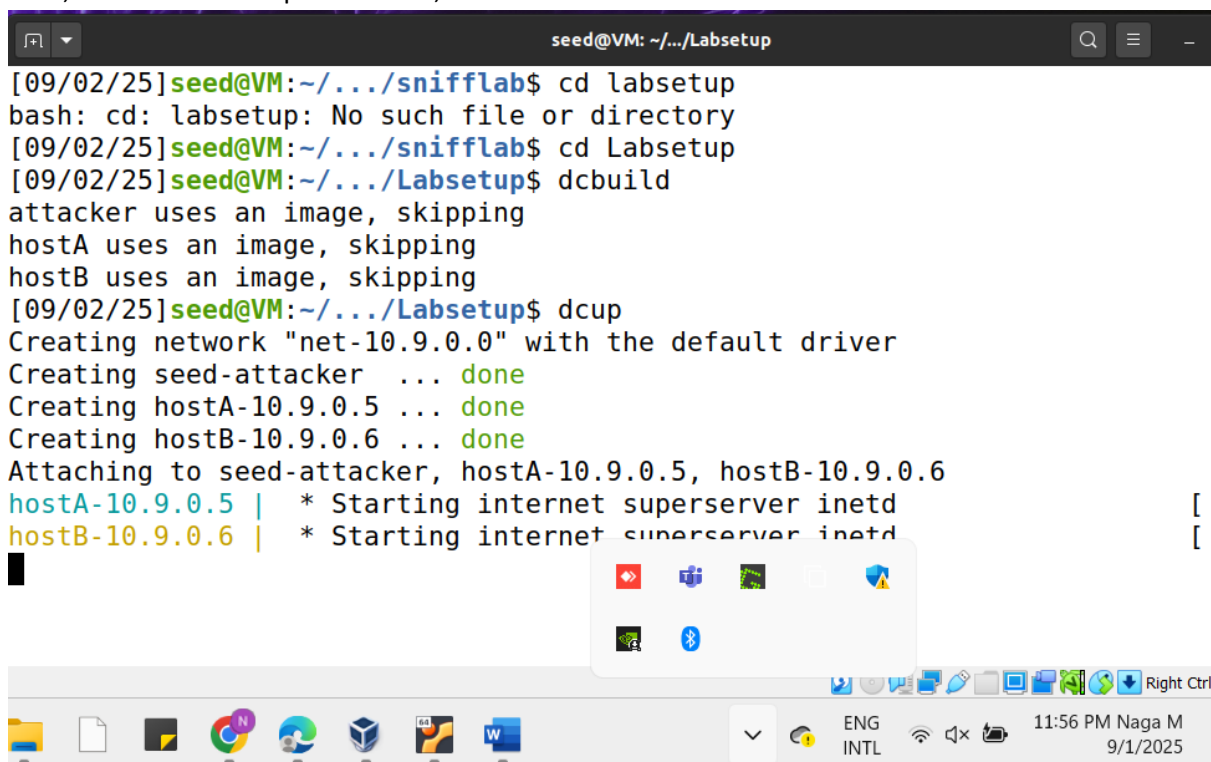
How to Download the lab setup zip file.>> click on the seed labs >> network security>> packet sniffing and spoofing.



Copy this zip to Documents(any other folder in VM), here I moved it to documents >> sniff lab>> there unzip the labsetup.zip file. From here right click and open the Terminal then we can directly move to Terminal

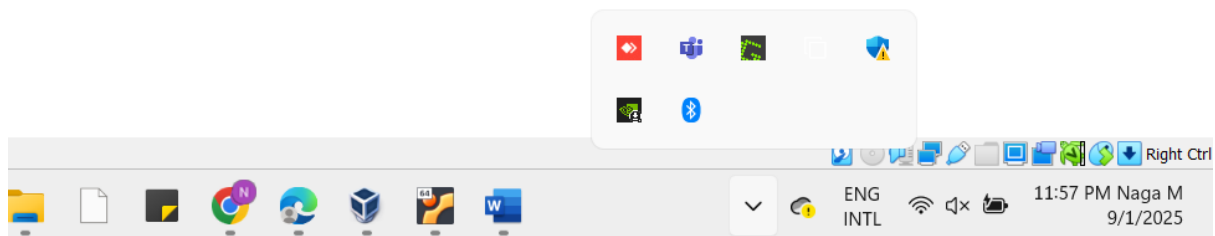


Now, We need to Set up Containers,



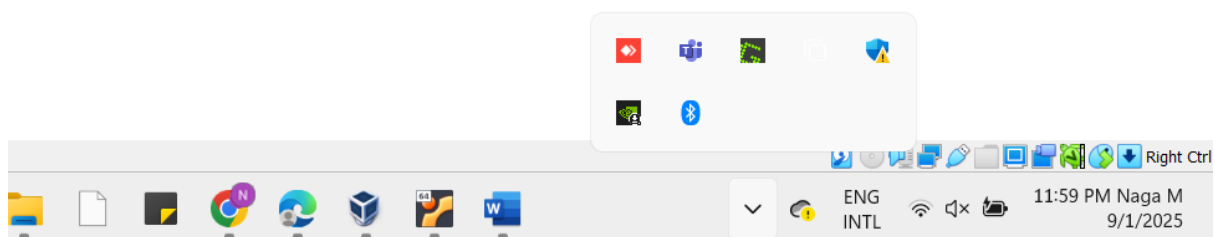
Open the another teminal in that type dockps

```
seed@VM: ~/.../Labsetup
[09/02/25] seed@VM:~/.../Labsetup$ dockps
39a95542f91e  hostB-10.9.0.6
d0d04ab88236  seed-attacker
a55182c1a150  hostA-10.9.0.5
[09/02/25] seed@VM:~/.../Labsetup$
```



Here we can see that the seed-attacker container and that id is d0, So we can run the command docksh d0

```
seed@VM: ~/.../Labsetup
[09/02/25] seed@VM:~/.../Labsetup$ dockps
39a95542f91e  hostB-10.9.0.6
d0d04ab88236  seed-attacker
a55182c1a150  hostA-10.9.0.5
[09/02/25] seed@VM:~/.../Labsetup$ docksh d0
root@VM:/#
```



After that we can enter that root.. we can check the ip address by using the ifconfig command

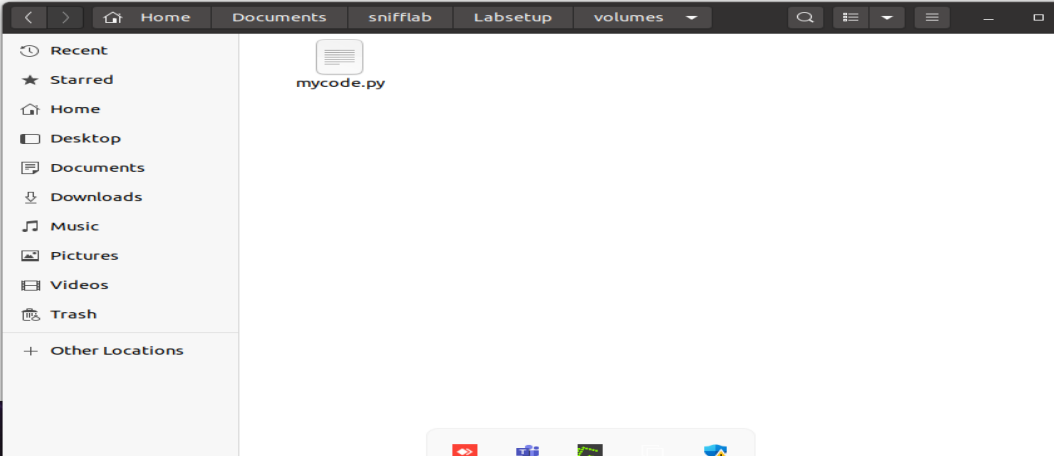
```
seed@VM: ~/.../Labsetup
[09/02/25] seed@VM: ~/.../Labsetup$ dockps
39a95542f91e  hostB-10.9.0.6
d0d04ab88236  seed-attacker
a55182c1a150  hostA-10.9.0.5
[09/02/25] seed@VM: ~/.../Labsetup$ docksh d0
root@VM:/# ifconfig
br-1d902ca0c748: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 10.9.0.1  netmask 255.255.255.0  broadcast 10.9.0.255
    inet6 fe80::42:f5ff:febf:28e8  prefixlen 64  scopeid 0x20<link>
    ether 02:42:f5:bf:28:e8  txqueuelen 0  (Ethernet)
    RX packets 0  bytes 0 (0.0 B)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 43  bytes 5196 (5.1 KB)
    TX errors 0  dropped 0 overruns 0  collisions 0

docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
    inet 172.17.0.1  netmask 255.255.255.255  broadcast 172.17.255.255
    ether 02:42:9c:48:00:12  txqueuelen 0  (Ethernet)
    RX packets 0  bytes 0 (0.0 B)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 0  bytes 0 (0.0 B)
    TX errors 0  dropped 0 overruns 0  collisions 0
```

LabTaskSet1: Using Scapy to Sniff and Spoof Packet

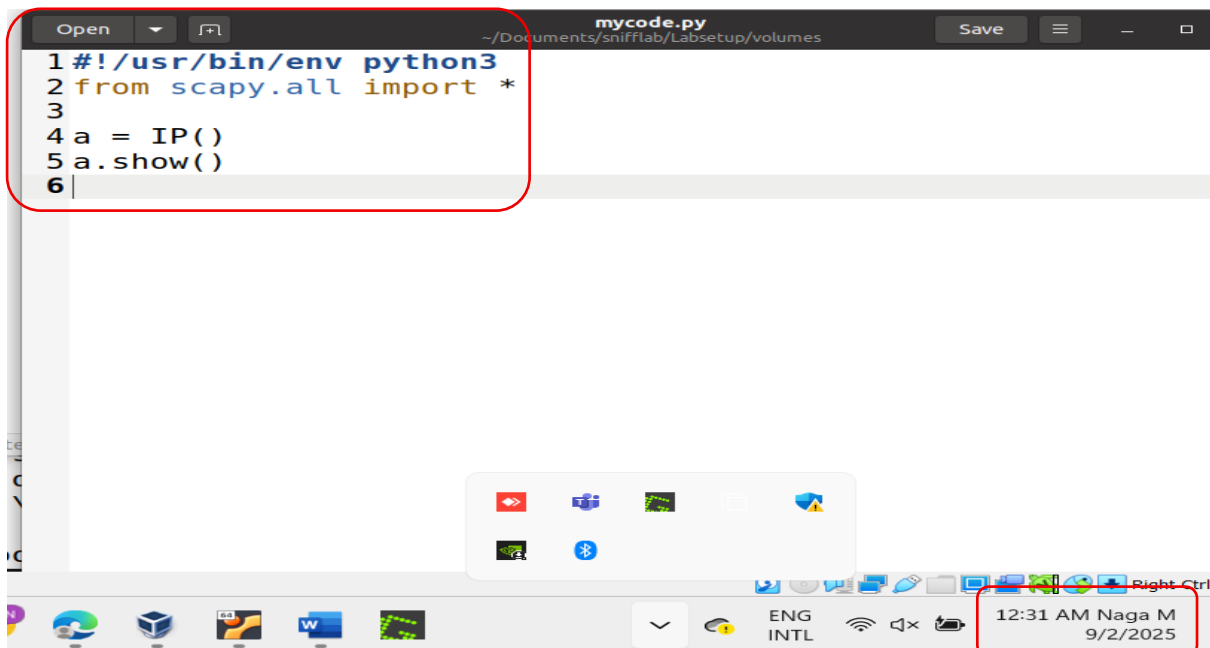
Create a .py file in that . For that open a new terminal and use the command touch mycode.py

```
seed@VM: ~/.../Labsetup
[09/02/25] seed@VM: ~/.../Labsetup$ cd volumes/
[09/02/25] seed@VM: ~/.../volumes$ touch mycode.py
[09/02/25] seed@VM: ~/.../volumes$
```



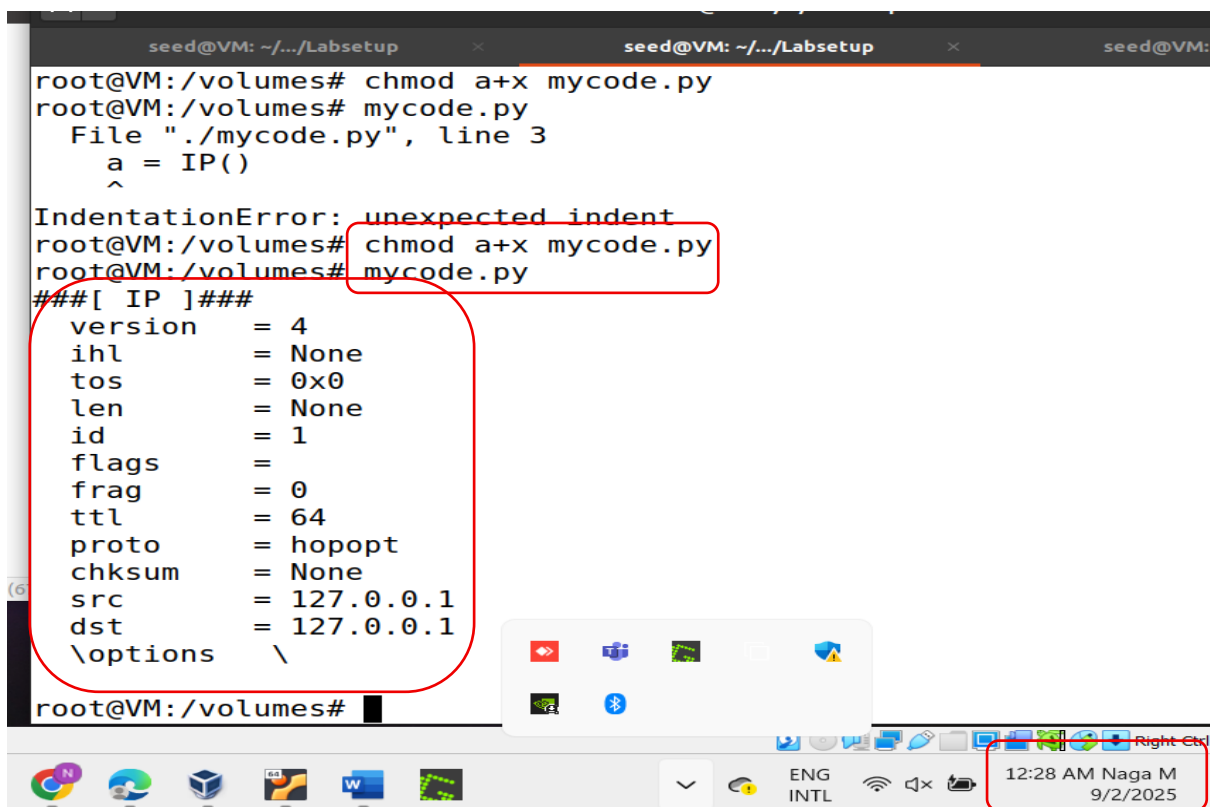
The screenshot shows a file manager window with a sidebar on the left containing 'Recent', 'Starred', 'Home', 'Desktop', 'Documents', 'Downloads', 'Music', 'Pictures', 'Videos', 'Trash', and 'Other Locations'. The main pane displays a single file named 'mycode.py' in the 'volumes' directory. The system tray at the bottom right shows the date and time as '12:10 AM Naga M 9/2/2025'.

After the command execution, we can immediately find a mycode.py file in the lab setup file. Double click on that file and write the python code and save it and close.



```
1#!/usr/bin/env python3
2from scapy.all import *
3
4a = IP()
5a.show()
6
```

Now open the root terminal and execute the mycode.py file, for that we need to use the chmod a+x mycode.py file, and then we can see the output of our system IP address details.



```
seed@VM: ~/.../Labsetup
root@VM:/volumes# chmod a+x mycode.py
root@VM:/volumes# mycode.py
File "./mycode.py", line 3
  a = IP()
  ^
IndentationError: unexpected indent
root@VM:/volumes# chmod a+x mycode.py
root@VM:/volumes# mycode.py
###[ IP ]###
version      = 4
ihl          = None
tos          = 0x0
len          = None
id           = 1
flags        =
frag         = 0
ttl          = 64
proto        = hopopt
chksum       = None
src          = 127.0.0.1
dst          = 127.0.0.1
\options     \

root@VM:/volumes#
```

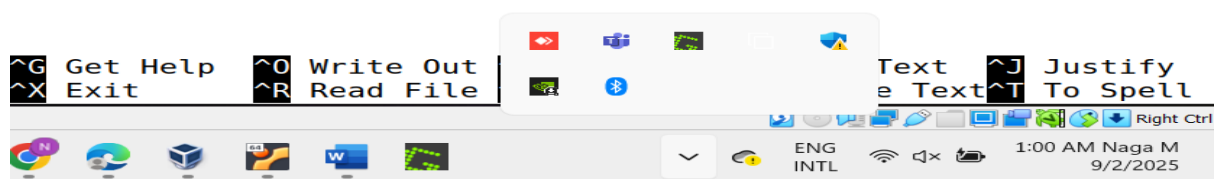
Task1.1:SniffingPacket

Create a sniffer.py file by using nano command and write the simple python code, and replace the interface id as we already seen in one of the above screenshots(ifconfig). Save the file.

```
seed@VM: ~/.../volumes
seed@VM: ~/.../Labsetup
GNU nano 4.8 sniffer.py
#!/usr/bin/env python3
from scapy.all import *

def print_pkt(pkt):
    pkt.show()

pkt = sniff(iface="br-1d902ca0c748", filter="icmp", prn=print_pkt)
```



```
seed@VM: ~/.../Labsetup
seed@VM: ~/.../Labsetup
seed@VM: ~/.../volumes

File "./mycode.py", line 3
  a = IP()
  ^
IndentationError: unexpected indent
root@VM:/volumes# chmod a+x mycode.py
root@VM:/volumes# mycode.py
###[ IP ]###
version    = 4
ihl        = None
tos        = 0x0
len        = None
id         = 1
flags      =
frag       = 0
ttl        = 64
proto      = hopopt
chksum     = None
src        = 127.0.0.1
dst        = 127.0.0.1
\options   \

root@VM:/volumes# chmod a+x sniffer.py
root@VM:/volumes# sniffer.py
```

Lets ping host B and in the root terminal execute the sniffer.py file.


```
[1]+  Stopped                  ping 10.9.0.6
[09/02/25]seed@VM:~/../volumes$ ping 10.9.0.6
PING 10.9.0.6 (10.9.0.6) 56(84) bytes of data:
64 bytes from 10.9.0.6: icmp_seq=1 ttl=64 time=0.139 ms
64 bytes from 10.9.0.6: icmp_seq=2 ttl=64 time=0.091 ms
64 bytes from 10.9.0.6: icmp_seq=3 ttl=64 time=0.050 ms
64 bytes from 10.9.0.6: icmp_seq=4 ttl=64 time=0.084 ms
64 bytes from 10.9.0.6: icmp_seq=5 ttl=64 time=0.036 ms
64 bytes from 10.9.0.6: icmp_seq=6 ttl=64 time=0.163 ms
64 bytes from 10.9.0.6: icmp_seq=7 ttl=64 time=0.085 ms
64 bytes from 10.9.0.6: icmp_seq=8 ttl=64 time=0.089 ms
64 bytes from 10.9.0.6: icmp_seq=9 ttl=64 time=0.099 ms
64 bytes from 10.9.0.6: icmp_seq=10 ttl=64 time=0.089 ms
64 bytes from 10.9.0.6: icmp_seq=11 ttl=64 time=0.084 ms
64 bytes from 10.9.0.6: icmp_seq=12 ttl=64 time=0.082 ms
64 bytes from 10.9.0.6: icmp_seq=13 ttl=64 time=0.115 ms
64 bytes from 10.9.0.6: icmp_seq=14 ttl=64 time=0.087 ms
64 bytes from 10.9.0.6: icmp_seq=15 ttl=64 time=0.174 ms
64 bytes from 10.9.0.6: icmp_seq=16 ttl=64 time=0.112 ms
64 bytes from 10.9.0.6: icmp_seq=17 ttl=64 time=0.212 ms
64 bytes from 10.9.0.6: icmp_seq=18 ttl=64 time=0.235 ms
64 bytes from 10.9.0.6: icmp_seq=19 ttl=64 time=0.066 ms
64 bytes from 10.9.0.6: icmp_seq=20 ttl=64 time=0.088 ms
64 bytes from 10.9.0.6: icmp_seq=21 ttl=64 time=0.062 ms
64 bytes from 10.9.0.6: icmp_seq=22 ttl=64 time=0.062 ms
64 bytes from 10.9.0.6: icmp_seq=23 ttl=64 time=0.062 ms
64 bytes from 10.9.0.6: icmp_seq=24 ttl=64 time=0.062 ms
64 bytes from 10.9.0.6: icmp_seq=25 ttl=64 time=0.062 ms
```

```
#su seed
$sniffer.py
```

```

seed@VM: ~/.../Labsetup
chksum      = 0xf0f5
id          = 0x1
seq         = 0x42
###[ Raw ]###
load        = '\x95\x89\xb6h\x00\x00\x00\x00\xf5\x01\x0f\x00\x00\x00\x0
0\x00\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f !"#%&\'()
*+,-./01234567'

exit
^Z
[1] Stopped sniffer.py
root@VM:/volumes# su seed
seed@VM:/volumes$ ./sniffer.py
Traceback (most recent call last):
  File "./sniffer.py", line 8, in <module>
    pkt = sniff(iface="br-1d902ca0c748", filter="icmp", prn=print_pkt)
  File "/usr/local/lib/python3.8/dist-packages/scapy/sendrecv.py", line 1036, in
    sniff
    sniffer._run(*args, **kwargs)
  File "/usr/local/lib/python3.8/dist-packages/scapy/sendrecv.py", line 906, in
    _run
    sniff_sockets[L2socket(type=ETH_P_ALL, iface=iface,
  File "/usr/local/lib/python3.8/dist-packages/scapy/arch/linux.py", line 398, i
    n __init__
    self.ins = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.htons(ty
e)) # noqa: E501
  File "/usr/lib/python3.8/socket.py", line 231, in
    socket.socket.__init__(self, family, type, proto)
PermissionError: [Errno 1] Operation not permitted
seed@VM:/volumes$

```

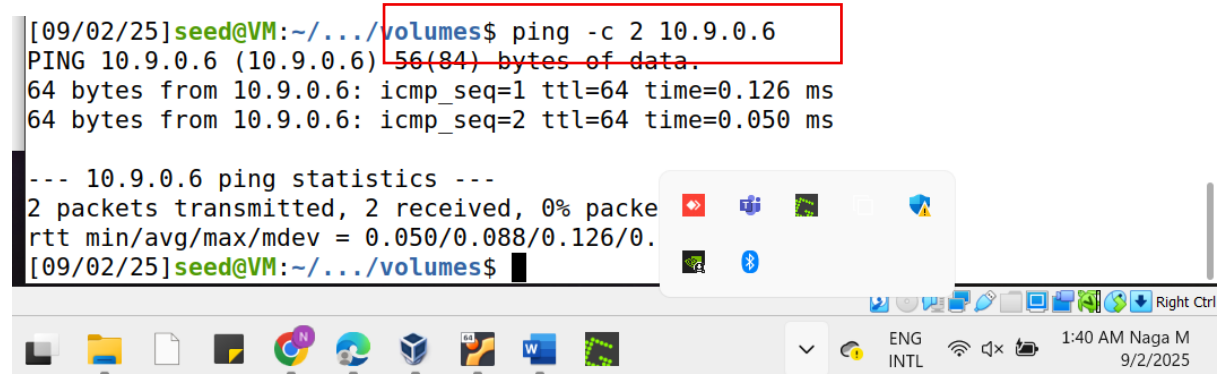
My observation: Lets see the output, as we got a permission Error and its saying that Operation not permitted. Because these kinds of sniffing tasks need root privileges without that we are not execute. Normal or Regular users doesn't have access to perform these kinds of tasks because of security reasons and these tasks allow direct access to the packets on the network.

Task 1.1B:

Capture only the ICMP packet : Here, I just tried to send only Two packets. The python program was same as the above.

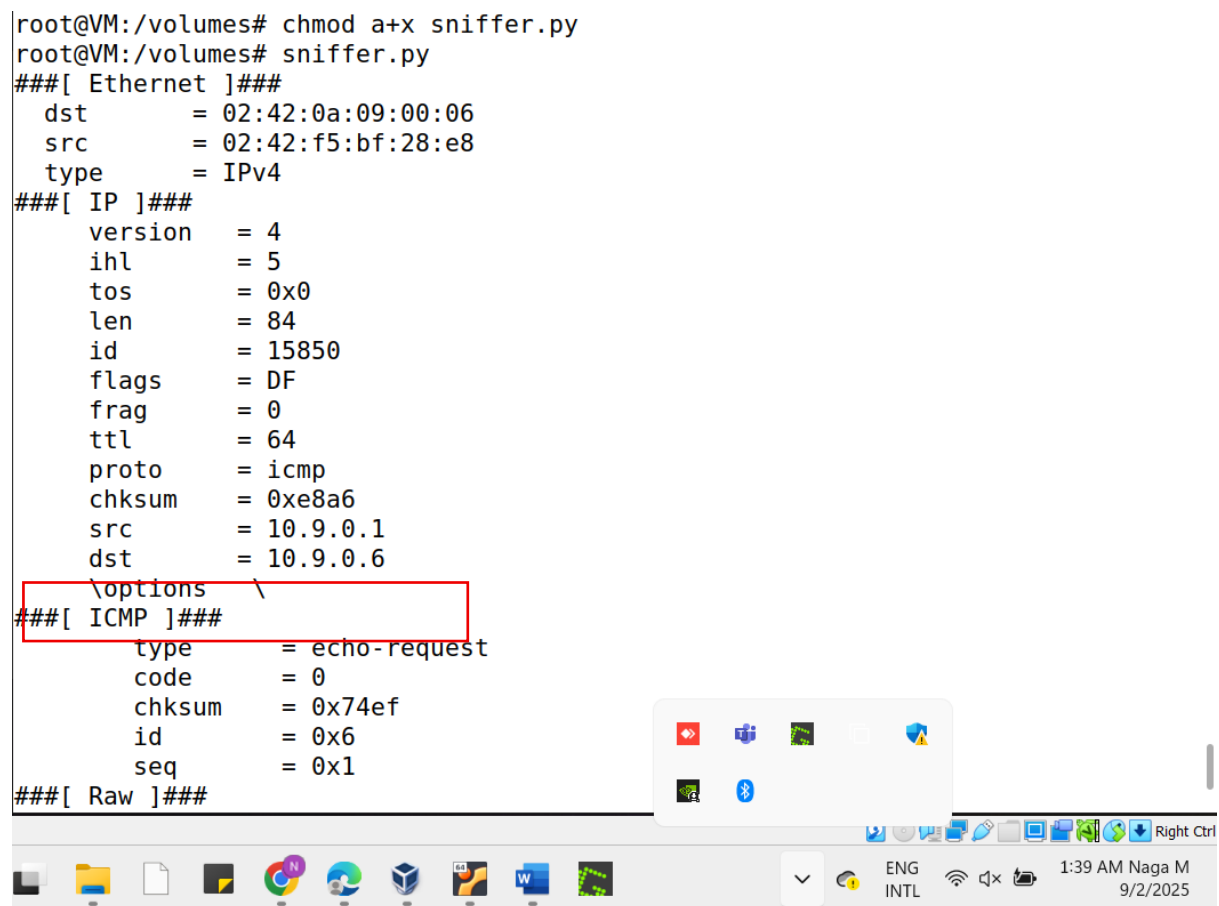
```
[09/02/25]seed@VM:~/.../volumes$ ping -c 2 10.9.0.6
PING 10.9.0.6 (10.9.0.6) 56(84) bytes of data:
64 bytes from 10.9.0.6: icmp_seq=1 ttl=64 time=0.126 ms
64 bytes from 10.9.0.6: icmp_seq=2 ttl=64 time=0.050 ms

--- 10.9.0.6 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time=100ms
rtt min/avg/max/mdev = 0.050/0.088/0.126/0.038 ms
[09/02/25]seed@VM:~/.../volumes$
```

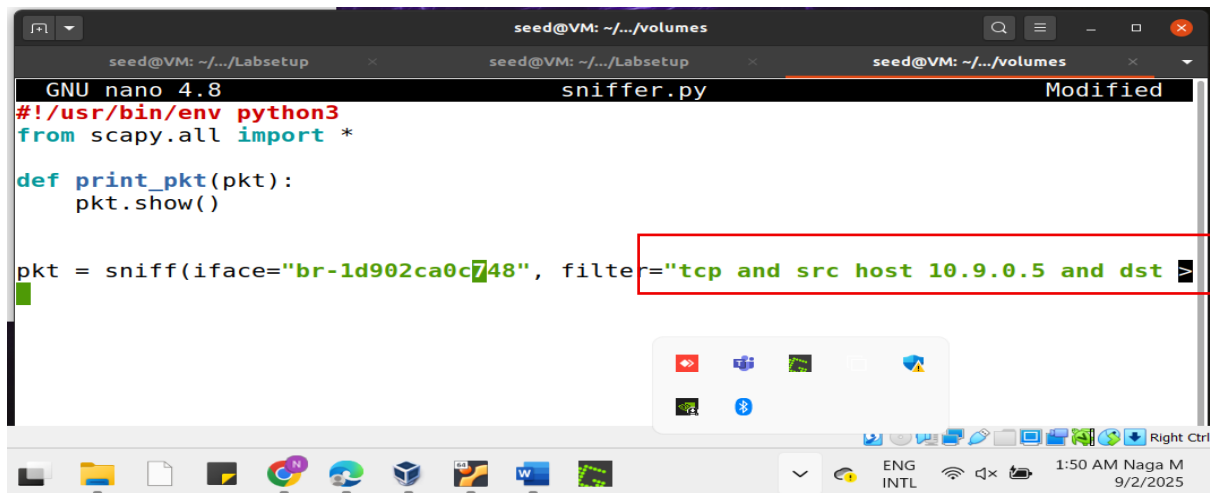


The Output here we see only ICMP packets are received

```
root@VM:/volumes# chmod a+x sniffer.py
root@VM:/volumes# sniffer.py
###[ Ethernet ]###
  dst      = 02:42:0a:09:00:06
  src      = 02:42:f5:bf:28:e8
  type     = IPv4
###[ IP ]###
  version  = 4
  ihl      = 5
  tos      = 0x0
  len      = 84
  id       = 15850
  flags    = DF
  frag     = 0
  ttl      = 64
  proto    = icmp
  checksum = 0xe8a6
  src      = 10.9.0.1
  dst      = 10.9.0.6
  \options \
###[ ICMP ]###
  type     = echo-request
  code     = 0
  checksum = 0x74ef
  id       = 0x6
  seq      = 0x1
###[ Raw ]###
```



2. Capture any TCP packet that comes from a particular IP and with a destination port number 23:
Open the sniffer.py file and edit the filter as "tcp and src host 10.9.0.5 and dst port 23"



```
seed@VM: ~/.../volumes
GNU nano 4.8 sniffer.py Modified
#!/usr/bin/env python3
from scapy.all import *

def print_pkt(pkt):
    pkt.show()

pkt = sniff(iface="br-1d902ca0c748", filter="tcp and src host 10.9.0.5 and dst
```

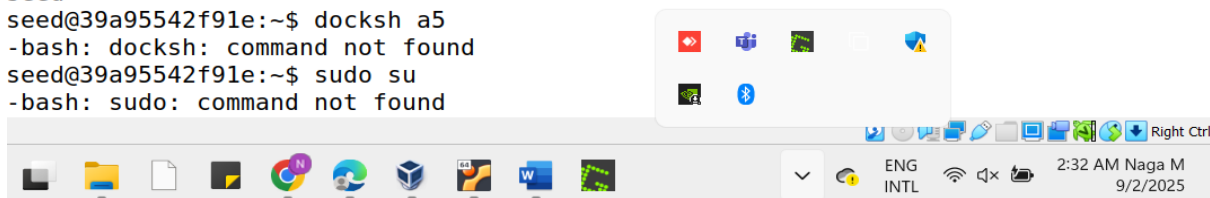
Here, we can run the command telnet, and see the output that the connection was established and we got the login to seed. Connection was successfully established means port 23 was open.

```
^Z
[7]+  Stopped                  ping 10.9.0.6
[09/02/25]seed@VM:~/.../volumes$ dockps
39a95542f91e  hostB-10.9.0.6
d0d04ab88236  seed-attacker
a55182c1a150  hostA-10.9.0.5
[09/02/25]seed@VM:~/.../volumes$ docksh a5
root@a55182c1a150:/# telnet 10.9.0.6 23
Trying 10.9.0.6...
Connected to 10.9.0.6.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
39a95542f91e login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage
```

This system has been minimized by removing packages and content that are not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Tue Sep 2 07:24:14 UTC 2025 from 10.9.0.1 on pts/1
seed@39a95542f91e:~\$ whoami
seed
seed@39a95542f91e:~\$ docksh a5
-bash: docksh: command not found
seed@39a95542f91e:~\$ sudo su
-bash: sudo: command not found




```
[09/02/25]seed@VM:~/../volumes$ docksh a5
root@a55182c1a150:/# ping -c 3 128.230.1.1
PING 128.230.1.1 (128.230.1.1) 56(84) bytes of data.
```

```
--- 128.230.1.1 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss
```

```
root@a55182c1a150:/#
```

And From the attacker VM, we can see the output.

```
./sniffer.py: line 4: `def print_pkt(pkt):'
root@VM:/volumes# chmod a+x sniffer.py
root@VM:/volumes# sniffer.py

###[ Ethernet ]###
  dst      = 02:42:f5:bf:28:e8
  src      = 02:42:0a:09:00:05
  type     = IPv4
###[ IP ]###
  version  = 4
  ihl      = 5
  tos      = 0x0
  len      = 84
  id       = 40768
  flags    = DF
  frag     = 0
  ttl      = 64
  proto    = icmp
  checksum = 0xf74
  src      = 10.9.0.5
  dst      = 128.230.1.1
  \options \
###[ ICMP ]###
  type     = echo-request
  code     = 0
  checksum = 0xba0f
  id       = 0x2f
  seq      = 0x1
###[ Raw ]###
```

Task 1.2: Spoofing ICMP Packets

```
seed@VM: ~/.../volumes
GNU nano 4.8 spoof.py
#!/usr/bin/env python3
from scapy.all import *

a = IP()
a.dst = "10.9.0.6"      # target
b = ICMP()
p = a/b
send(p)
```

Lets execute the spoof.py file with the command chmod a+x spoof.py.

```
root@VM:/volumes# chmod a+x spoof.py
root@VM:/volumes# spoof.py
.
```

Sent 1 packets.

```
root@VM:/volumes#
```

Here we that the packet was sent, so then open the wireshark and in filter select the ICMP and interface address then we can see the output as follows.

[SEED Labs] Capturing from br-1d902ca0c748

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	2025-09-02 04:0...	02:42:f5:bf:28:e8	Broadcast	ARP	42	Who has 10.9.0.6? Tell 1
2	2025-09-02 04:0...	02:42:0a:09:00:06	02:42:f5:bf:28:e8	ARP	42	10.9.0.6 is at 02:42:0a:09:00:06
3	2025-09-02 04:0...	10.9.0.1	10.9.0.6	ICMP	42	Echo (ping) request id=
4	2025-09-02 04:0...	10.9.0.6	10.9.0.1	ICMP	42	Echo (ping) reply id=
5	2025-09-02 04:0...	02:42:0a:09:00:06	02:42:f5:bf:28:e8	ARP	42	Who has 10.9.0.1? Tell 1
6	2025-09-02 04:0...	02:42:f5:bf:28:e8	02:42:0a:09:00:06	ARP	42	10.9.0.1 is at 02:42:f5:bf:28:e8

Frame 1: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface br-1d902ca0c748, id 0

Ethernet II, Src: 02:42:f5:bf:28:e8 (02:42:f5:bf:28:e8), Dst: Broadcast (ff:ff:ff:ff:ff:ff)

Address Resolution Protocol (request)

```
0000  ff ff ff ff ff ff 02 42 f5 bf 28 e8 08 06 00 01  ....B..(-...
0010  08 00 06 04 00 01 02 42 f5 bf 28 e8 0a 09 00 01  ....B..(-...
0020  00 00 00 00 00 00 0a 09 00 06  ....
```

br-1d902ca0c748: <live capture in progress>

5 (100.0%) Profile: Default

ENG INTL 3:07 AM Naga M 9/2/2025